

NetShield AI

Network Anomaly Detection & Threat Monitoring System — Project Documentation

1. Project Overview

NetShield AI is an AI-powered network security platform that monitors live network traffic, classifies packet flows using a trained machine learning model, and surfaces threats through a real-time analyst dashboard.

Problem Statement

Security teams need continuous visibility into network traffic to catch intrusions (DDoS, port scans, botnet activity) before they escalate, but manual traffic review does not scale.

Objective

Provide a centralized platform where security analysts can capture live packets, get AI-driven risk classification per flow, review alerts and analytics, and manage detection history — with role-based access control.

Key Features Implemented

- JWT-based authentication with role-based access control (Admin, Security Analyst)
- Live packet capture and monitoring (interface-level capture with simulate-traffic option)
- Real-time AI predictions per traffic flow (attack type, confidence, risk score, threat level)
- Security analytics dashboard (attack timeline, attack/protocol distribution, risk trend)
- Detection history, alerts, and reporting views
- User management with role assignment and account status control

2. System Architecture & Technology Stack

Flow: User → React (Vite) Frontend → FastAPI Backend (Auth/RBAC) → Random Forest ML Model + Live Packet Capture → MongoDB.

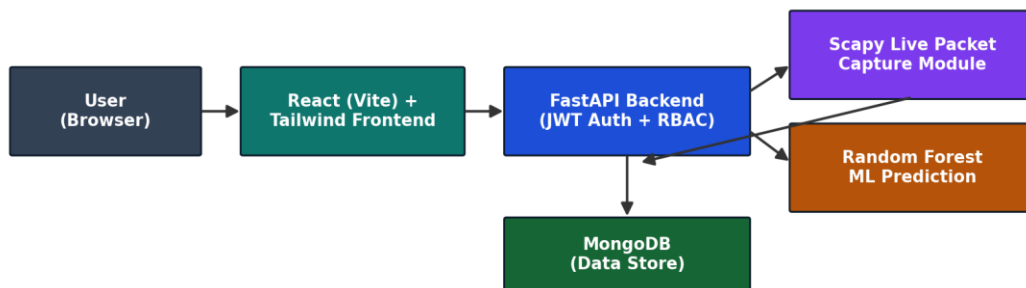


Figure 1: NetShield AI system architecture (as implemented).

Technology	Purpose
React (Vite) + Tailwind CSS	Single-page analyst frontend — dashboard, monitoring, analytics, user management UI
FastAPI (Python)	Backend REST API — auth, packet/prediction endpoints, alerts, analytics, reports
MongoDB	Primary data store for users, packets/flows, predictions, alerts
Scikit-learn (Random Forest)	Trained classification model for traffic flow / attack-type prediction
Scapy	Live network packet capture on the host interface
JWT	Stateless authentication and session tokens

Technology	Purpose
Docker / Docker Compose	Containerized backend and frontend services for deployment (see docker-compose.yml)

Note: the original specification proposed a broader stack (Next.js, PostgreSQL, TensorFlow, XGBoost, Wireshark, Zeek, SIEM integrations). Only the items in the table above are confirmed as implemented — see the Technology Verification table in Section 6.

3. Main Modules & Implementation

Authentication & User Management

Purpose: control who can access the platform and what they can do. Implementation: JWT-based login/registration with role-based access (Admin, Security Analyst); the User Management screen lets an Admin view all accounts, change roles, and disable/delete users. Output: authenticated sessions and an auditable user/role list (Figure 5).

Dashboard

Purpose: give an at-a-glance view of network security posture. How it works: aggregates packet, threat, and alert counts and shows live-updating tiles (total packets, threats detected, active/critical alerts), model status, and dataset status. Output: a real-time operations overview (Figure 2).

Packet Monitoring

Purpose: capture and inspect live network traffic. How it works: starts/stops live capture on the host network interface (or a simulate-traffic mode for demos), feeds each captured flow to the ML model, and lists live AI predictions with source IP, attack type, confidence, risk percentage, and threat level. Output: a continuously updating table of classified traffic (Figure 3).

AI Prediction / Threat Classification

Purpose: classify each traffic flow as normal or a specific attack type. How it works: a trained Random Forest model scores each flow and returns an attack label (e.g., normal, ddos, portscan, botnet) with a confidence percentage, which is mapped to a Low/Medium/Critical threat level. Output: per-flow prediction records shown in Packet Monitoring and summarized on the Dashboard and Analytics pages.

Security Analytics

Purpose: summarize detection performance and traffic patterns over time. How it works: computes detection rate, alert resolution rate, total predictions, and false positives, and renders an attack timeline, attack-type distribution, protocol distribution, and 7-day risk trend as charts. Output: a analytics view for reviewing trends (Figure 4).

Alerts & Detection History

Purpose: surface actionable threats. How it works: high-risk predictions are logged as alerts/latest threats with source IP, attack type, and severity; the dashboard's Detection History panel lists recent prediction activity. Output: a running record analysts can review and act on.

4. Dataset & Machine Learning

The model was trained on network intrusion datasets (CICIDS2017 / UNSW-NB15), preprocessed into a feature set, and trained as a Random Forest classifier. [VERIFY: exact feature list and hyperparameters against the shipped training script.]



Figure 6: Machine learning training and prediction workflow.

Metric	Value
Training samples	178,465

Metric	Value
Testing samples	44,617
Accuracy	$\approx 99.99\%$
Precision	≈ 1.00
Recall	≈ 1.00
F1-score	≈ 1.00
Confusion matrix	[[18954, 0], [4, 25659]]

These numbers reflect performance on the held-out test split of the training dataset. They are a measure of how well the model fits this dataset and should not be read as a guarantee of equivalent accuracy on unseen, real-world network traffic, which is typically noisier and less balanced.

5. Application Screens & Results

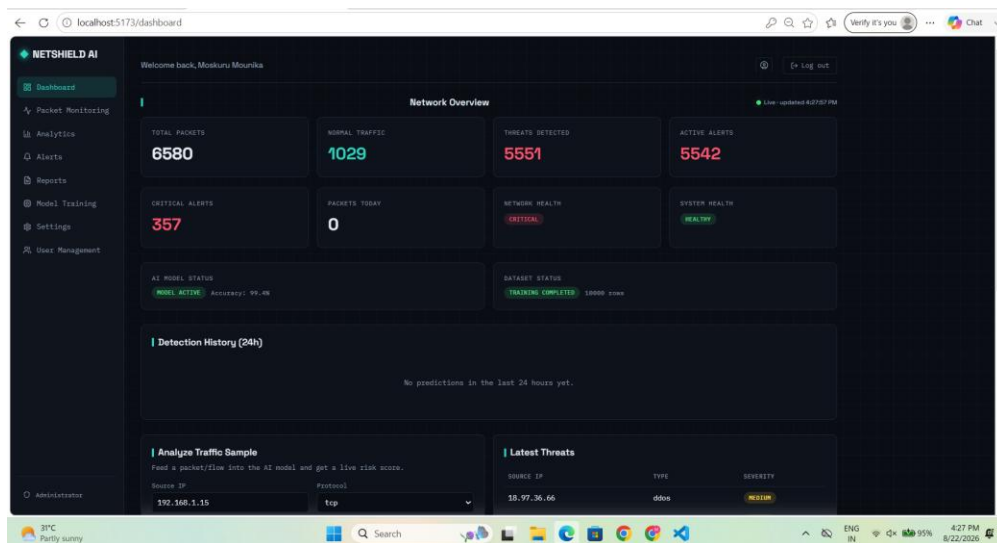


Figure 2: Dashboard — network overview, live tiles, AI model status, and detection history.

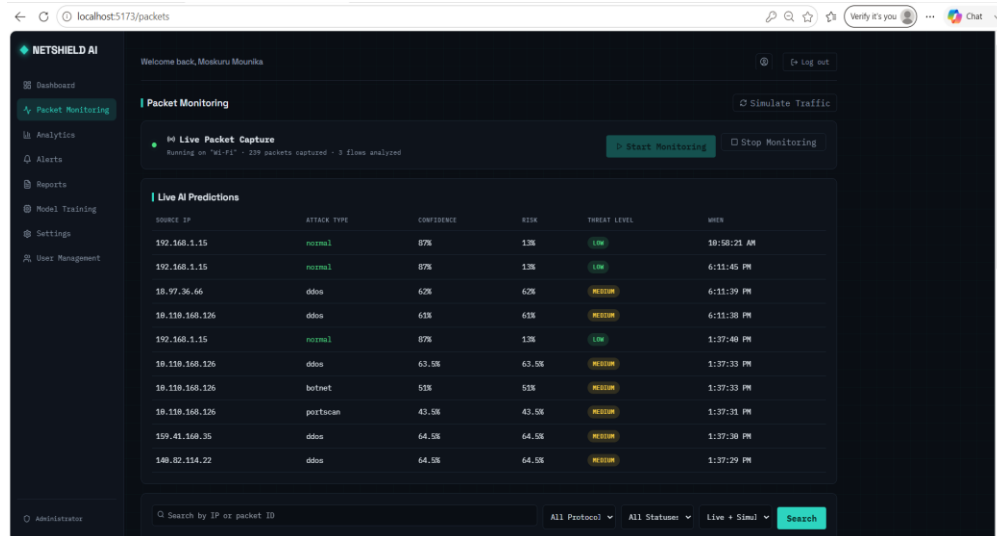


Figure 3: Packet Monitoring — live capture control and per-flow AI predictions (attack type, confidence, risk, threat level).

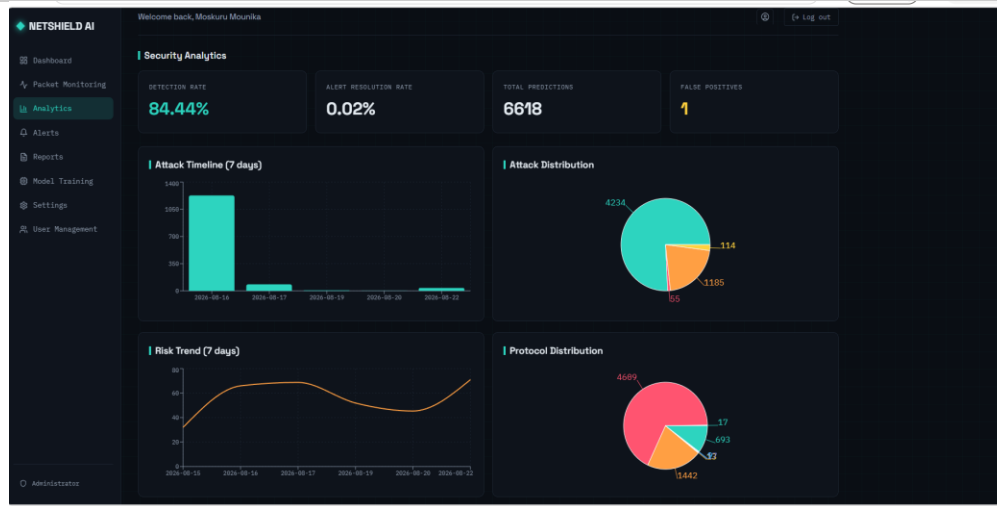


Figure 4: Security Analytics — detection rate, alert resolution rate, attack timeline, and protocol/attack distribution.

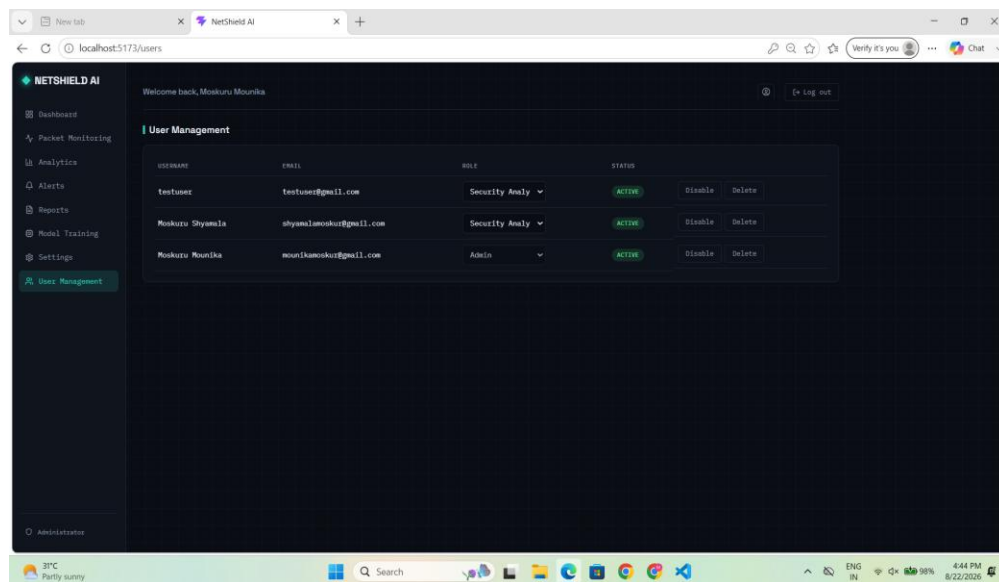


Figure 5: User Management — role assignment (Admin / Security Analyst) and account status control.

6. Testing, Deployment & Future Scope

Testing

- Authentication and role-based access testing across Admin / Security Analyst accounts
- API endpoint testing for packet, prediction, alert, and analytics routes
- ML model evaluation on the held-out test split (see Section 4 metrics)
- Manual UI testing of dashboard, monitoring, analytics, and user-management screens (see screenshots)

Deployment

The project ships with a docker-compose.yml defining a backend service (FastAPI, built from ./backend) and a frontend service (built from ./frontend, served on port 5173). Screenshots show the app running at localhost:5173, i.e. local Docker/dev deployment. [VERIFY: whether a cloud (AWS/Azure) deployment beyond local Docker was completed — no evidence of this was provided.]

Future Scope

- Deeper real-time packet monitoring across multiple interfaces/protocols
- Improved, model-driven risk scoring beyond raw confidence
- Broader attack-category coverage and periodic model retraining
- Email/notification-based alerting for critical threats

- Training on larger, more realistic real-world traffic datasets
- Production-scale cloud deployment and SIEM/threat-intelligence integrations

7. Conclusion

NetShield AI addresses the need for continuous, AI-assisted network traffic monitoring by combining a FastAPI backend, a React analytics frontend, and a Random Forest classifier trained on intrusion-detection datasets. The implemented platform supports authenticated, role-based access; live packet capture; per-flow threat classification; and analytics/reporting dashboards, running via Docker Compose. On its held-out test data the model achieves near-perfect accuracy, precision, and recall, though real-world performance would need validation on live traffic. The system provides a practical foundation for SOC-style monitoring, with clear next steps around real-time scale, alerting, and broader deployment.

Technology Verification

Technology	Actually Used?	Evidence
React (Vite) + Tailwind	Yes	UI screenshots; frontend/ build served via Docker on port 5173
FastAPI (Python)	Yes	docker-compose.yml backend service; API-driven dashboard/monitoring UI
MongoDB	Yes (per prior verified build)	[VERIFY against shipped backend/.env.example — actual DB connection not in this upload]
Scikit-learn / Random Forest	Yes	Model status "MODEL ACTIVE" + accuracy shown on dashboard; metrics in Section 4
Scapy (packet capture)	Yes (per prior verified build)	"Live Packet Capture" control + "Running on Wi-Fi" in Packet Monitoring screen
Docker / Docker Compose	Yes	docker-compose.yml defines backend + frontend services